

Phase 8: Full, Complete, and Strict Binary Tree Comparisons

0. Purpose of This Phase

Phase 8 teaches the difference between three binary tree terms that beginners often confuse:

- 1. **Full binary tree**
- 2. **Complete binary tree**
- 3. **Strict binary tree**

These names look similar, but they do **not** mean the same thing.

The main beginner idea is:

Strict checks child count. Full checks maximum packing. Complete checks array gaps.

In simple words:

Tree type	Main question
Strict binary tree	Does every node have either 0 or 2 children?
Full binary tree	Is the tree completely packed for its height?
Complete binary tree	Are there no missing gaps when stored level by level?

1. Current Progress Analysis

Before Phase 8, you already learned ordinary and strict binary trees.

Previous phase	What you learned	Why it matters in Phase 8
Phase 1	Root, child, leaf, internal node, degree, height	These words are needed to describe full, complete, and strict trees.
Phase 2	Binary tree height and node formulas	Full binary trees use the maximum-node formula.
Phase 3	Array representation of binary trees	Complete binary trees are easiest to understand using array positions.
Phase 4	Recursive traversal	Checker functions often use recursion.

Previous phase	What you learned	Why it matters in Phase 8
Phase 5	Level-order traversal and queues	Complete trees are filled level by level.
Phase 6	Counting nodes and height	Full-tree checking needs total nodes and height.
Phase 7	Strict binary trees	Phase 8 compares strict trees with full and complete trees.

So your current position is:

You already know how binary trees work. Now you are learning how to separate three similar terms: **strict**, **full**, and **complete**.

2. What Phase 8 Covers

Phase 8 covers these concepts:

1. Meaning of a full binary tree
2. Full binary tree formula
3. Meaning of a complete binary tree
4. Complete binary tree book definition
5. Complete binary tree array-gap rule
6. Why every full tree is complete
7. Why not every complete tree is full
8. Strict vs complete comparison
9. Naming confusion in textbooks
10. Four-case comparison:
11. strict and complete
12. strict but not complete
13. complete but not strict
14. neither strict nor complete
15. Full tree checker
16. Complete tree checker using array-style indices
17. Strict tree checker
18. Complete beginner C++ program
19. Dry run of the program
20. Common beginner mistakes
21. Final memory tricks
22. Practice questions

Part A: Key Definitions

3. Binary Tree Reminder

A **binary tree** is a tree where each node can have at most two children.

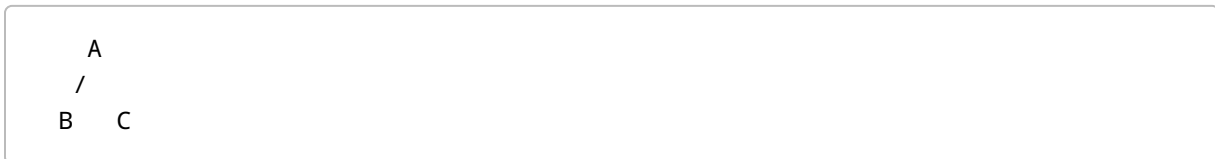
The two child positions are:

1. left child
2. right child

A node in a binary tree may have:

Number of children	Name
0 children	leaf node / external node
1 child	degree-1 node
2 children	degree-2 node

Example:



Here:

- A has 2 children.
- B has 0 children.
- C has 0 children.

This is a valid binary tree.

4. Height Convention Used in This Curriculum

In this curriculum, **height counts edges**.

A single root node has height 0.

Example 1:

A

Height = 0

Why?

Because there are no edges.

Example 2:

A
|
B

Height = 1

Why?

Because the longest path has 1 edge.

Example 3:

A
|
B
|
C

Height = 2

Why?

Because the longest path has 2 edges.

This matters because the full binary tree formula uses height starting from 0.

Part B: Full Binary Tree

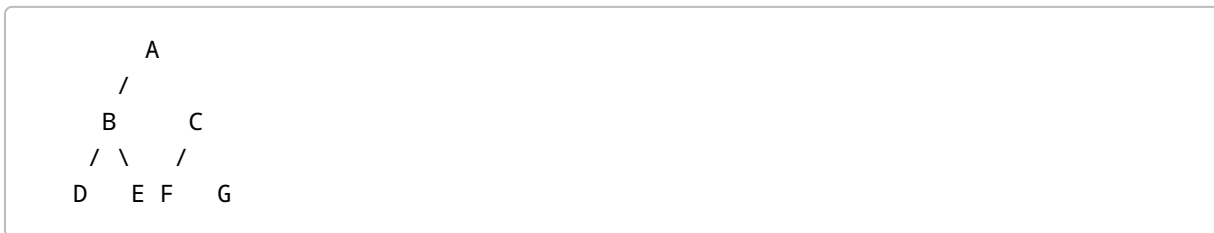
5. What Is a Full Binary Tree?

In this curriculum, a **full binary tree** means:

A binary tree that is completely packed with nodes for its height.

That means every level is fully filled.

Example:



This tree is full because:

- Level 0 has 1 node: A
- Level 1 has 2 nodes: B, C
- Level 2 has 4 nodes: D, E, F, G

Every possible position is filled.

6. Important Naming Note

Some books call this kind of tree a **perfect binary tree**.

In this curriculum:

Full binary tree = completely packed tree

In many other books:

Perfect binary tree = completely packed tree

So if another source says **perfect binary tree**, it may mean what this curriculum calls a **full binary tree**.

The important thing is not only the name.

The important thing is the rule:

Every level is completely filled.

7. Full Binary Tree Formula

If a full binary tree has height H , then the number of nodes is:

$$N = 2^{(H + 1)} - 1$$

Where:

- N = number of nodes
- H = height of the tree

8. Why the Formula Uses $H + 1$

Height starts at 0.

If height is 2, the levels are:

```
Level by height:
0
1
2
```

That is 3 levels.

So:

$$\text{number of levels} = H + 1$$

A full binary tree doubles the number of nodes at each level:

Height level	Number of nodes
0	1
1	2
2	4

Total:

$$1 + 2 + 4 = 7$$

Formula:

$$2^{(H + 1)} - 1$$

For $H = 2$:

$$N = 2^{(2 + 1)} - 1$$

$$N = 2^3 - 1$$

$$N = 8 - 1$$

$$N = 7$$

So a full binary tree of height 2 has 7 nodes.

9. Full Binary Tree Formula Table

Height H	Full tree nodes: $2^{(H + 1)} - 1$
0	1
1	3
2	7
3	15
4	31
5	63

10. Full Binary Tree Simple Memory Trick

Full = every level is full.

Or:

Full = maximum nodes for that height.

Part C: Complete Binary Tree

11. What Is a Complete Binary Tree?

A **complete binary tree** does not have to be fully packed.

A complete binary tree means:

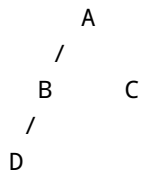
All levels before the last level are full, and the last level is filled from left to right.

Another simple definition:

A complete binary tree has no gaps when stored level by level in an array.

12. Complete Binary Tree Example

Example:



This tree is complete.

Why?

Because nodes are filled from left to right:

Index:	1	2	3	4
Node:	A	B	C	D

There is no missing position before D.

The tree is not fully packed, but it is still complete.

13. Complete Binary Tree Book Definition

A complete binary tree is usually described like this:

Every level is completely filled except possibly the last level, and the last level is filled from left to right.

This means:

- The first level must be full.
- The second level must be full before using the third level.
- The last level may be partially filled.
- But the last level must fill from left to right.

14. Complete Binary Tree as Array Layout

Complete trees are strongly connected to array representation.

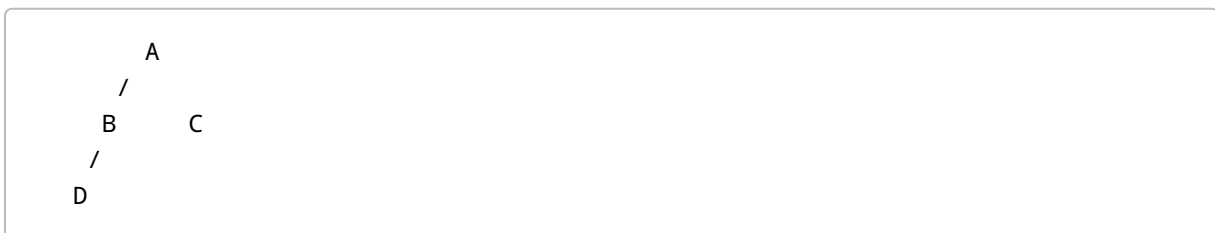
In array representation, we store nodes level by level from left to right.

The root starts at index 1.

Index formulas:

Relationship	Formula
Left child of index i	$2 * i$
Right child of index i	$2 * i + 1$
Parent of index i	$i / 2$

Example:



Array positions:

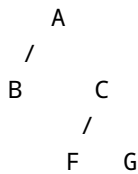
```
A = index 1  
B = index 2
```

```
C = index 3
D = index 4
```

Since the filled positions are 1, 2, 3, 4 with no gap, the tree is complete.

15. Not Complete Binary Tree Example

Example:



Assign array-style indices:

```
A = 1
B = 2
C = 3
F = 6
G = 7
```

Positions 4 and 5 are missing.

But positions 6 and 7 exist.

That means there are gaps.

So this tree is **not complete**.

16. Complete Binary Tree Simple Memory Trick

Complete = no gaps.

Or:

Complete = fill level by level, left to right.

Part D: Strict Binary Tree Review

17. What Is a Strict Binary Tree?

A **strict binary tree** is a binary tree where every node has either:

- 0 children
- or 2 children

A node with exactly 1 child is not allowed.

Simple definition:

Strict binary tree = every node has 0 or 2 children.

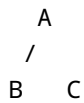
Another name for a strict binary tree is:

proper binary tree

So:

Strict binary tree = Proper binary tree

18. Strict Binary Tree Example



Check each node:

Node	Number of children	Valid in strict tree?
A	2	Yes
B	0	Yes
C	0	Yes

So this tree is strict.

19. Not Strict Binary Tree Example

```
  A
 /
B
```

Check each node:

Node	Number of children	Valid in strict tree?
A	1	No
B	0	Yes

Node A has exactly one child.

So the tree is not strict.

20. Strict Binary Tree Simple Memory Trick

One lonely child breaks strictness.

Part E: Full vs Complete

21. Full and Complete Are Related

A full binary tree is always complete.

Why?

Because if every level is fully packed, there cannot be gaps.

Example full tree:

```
  A
 / \
B   C
```

```

      / \ /
     D  E F  G

```

Array:

Index:	1	2	3	4	5	6	7
Node:	A	B	C	D	E	F	G

No gaps exist.

So it is complete.

22. Complete Is Not Always Full

A complete tree may have a partially filled last level.

Example:

```

      A
     / \
    B   C
   /
  D

```

This tree is complete because the filled positions are:

1, 2, 3, 4

There are no gaps.

But it is not full because height is 2.

A full tree of height 2 must have:

$2^{(2 + 1)} - 1 = 7$ nodes

This tree has only 4 nodes.

So:

Complete? Yes
Full? No

23. Full vs Complete Summary

Tree type	Rule	Relationship
Full	Every level is 100% packed	Always complete
Complete	No gaps in level-order array layout	Not always full

Memory trick:

Full = completely filled.
Complete = gap-free.

Part F: Strict vs Complete

24. Strict and Complete Check Different Things

Strict checks child count.

Complete checks array layout.

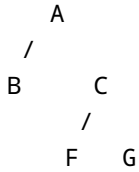
Concept	What it checks
Strict	Does every node have 0 or 2 children?
Complete	Are there no gaps in level-order array positions?

So a tree can be:

- strict and complete
- strict but not complete
- complete but not strict
- neither strict nor complete

25. Strict Does Not Mean Complete

Example:



Strict check:

Node	Children	Strict rule satisfied?
A	B and C	Yes
B	none	Yes
C	F and G	Yes
F	none	Yes
G	none	Yes

So it is strict.

Complete check using indices:

A = 1
B = 2
C = 3
F = 6
G = 7

Positions 4 and 5 are missing.

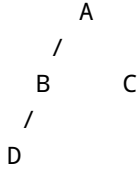
So it is not complete.

Conclusion:

Strict? Yes
Complete? No

26. Complete Does Not Mean Strict

Example:



Complete check:

```
A = 1
B = 2
C = 3
D = 4
```

Positions 1, 2, 3, 4 are filled without gaps.

So it is complete.

Strict check:

Node B has exactly one child: D.

A node with one child is not allowed in a strict binary tree.

So it is not strict.

Conclusion:

```
Complete? Yes
Strict? No
```

27. Strict vs Complete Memory Trick

```
Strict = child-count rule.
Complete = no-gap rule.
```

Or:

Strict asks: 0 or 2 children?
Complete asks: any missing array positions?

Part G: Naming Confusion

28. Why the Names Can Be Confusing

Some textbooks use names differently.

For example, some notes may use **complete binary tree** to mean what other books call **strict** or **proper** binary tree.

This curriculum separates the terms like this:

Curriculum term	Meaning	Other common name
Strict binary tree	Every node has 0 or 2 children	Proper binary tree
Full binary tree	Every level is completely packed	Perfect binary tree in many books
Complete binary tree	No gaps; last level fills left to right	Almost-complete binary tree in some books

The safest approach is:

Do not rely only on the name. Always state the rule.

Part H: Four-Case Comparison

29. Case 1: Strict and Complete

Example:

```
  A
 /
B  C
```

Strict check:

- A has 2 children.
- B has 0 children.
- C has 0 children.

So it is strict.

Complete check:

```
A = 1
B = 2
C = 3
```

No gaps.

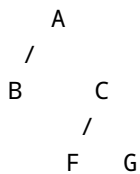
So it is complete.

Final answer:

```
Strict? Yes
Complete? Yes
```

30. Case 2: Strict but Not Complete

Example:



Strict check:

- A has 2 children.
- B has 0 children.
- C has 2 children.
- F has 0 children.
- G has 0 children.

So it is strict.

Complete check:

```
A = 1
B = 2
C = 3
F = 6
G = 7
```

Positions 4 and 5 are missing.

So it is not complete.

Final answer:

```
Strict? Yes
Complete? No
```

31. Case 3: Complete but Not Strict

Example:

```
      A
     /
    B   C
   /
  D
```

Complete check:

```
A = 1
B = 2
C = 3
D = 4
```

No gaps.

So it is complete.

Strict check:

B has exactly one child.

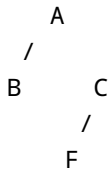
So it is not strict.

Final answer:

Strict? No
Complete? Yes

32. Case 4: Neither Strict Nor Complete

Example:



Strict check:

C has exactly one child: F.

So it is not strict.

Complete check:

A = 1
B = 2
C = 3
F = 6

Positions 4 and 5 are missing before position 6 appears.

So it is not complete.

Final answer:

Strict? No
Complete? No

33. Four-Case Summary Table

Case	Strict?	Complete?	Why?
Full packed triangle	Yes	Yes	Every internal node has 2 children and no array gaps exist.
Strict but bottom leaves only on one side causing gaps	Yes	No	Child counts are valid, but array positions skip spaces.
A with B and C, and B has only left child D	No	Yes	B has one child, but positions 1, 2, 3, 4 are gap-free.
Single-child nodes on the right with gaps	No	No	It breaks strictness and creates array gaps.

Part I: Checker Functions

34. Why We Need Checker Functions

A checker function answers a yes/no question.

For this phase, we want functions that answer:

1. Is this tree full?
2. Is this tree strict?
3. Is this tree complete?

In C++, these functions usually return `bool` :

true
false

35. Node Structure Used in the Code

```
struct Node {  
    char data;  
    Node* left;  
    Node* right;  
  
    Node(char val) : data(val), left(nullptr), right(nullptr) {}  
};
```

Explanation:

Part	Meaning
char data	stores the value of the node
Node* left	stores the address of the left child
Node* right	stores the address of the right child
nullptr	means no child exists there

36. Counting Nodes

To check whether a tree is full, we need to count total nodes.

Code:

```
int countNodes(Node* root) {  
    if (root == nullptr) return 0;  
    return 1 + countNodes(root->left) + countNodes(root->right);  
}
```

Meaning:

Total nodes = current node + left subtree nodes + right subtree nodes

Memory trick:

Count = 1 + left + right

37. Calculating Height in Edges

Full-tree formula uses height in edges.

Code:

```
int heightEdges(Node* root) {  
    if (root == nullptr) return -1;  
    return 1 + max(heightEdges(root->left), heightEdges(root->right));  
}
```

Why return `-1` for `nullptr`?

Because a single node should have height 0.

For a leaf node:

```
left height = -1  
right height = -1  
height = 1 + max(-1, -1)  
height = 0
```

So a leaf gets height 0, which matches the curriculum convention.

38. Full Tree Checker

A tree is full if:

```
actual number of nodes = 2^(height + 1) - 1
```

Code:

```
bool isFullPacked(Node* root) {  
    int total = countNodes(root);  
    int h = heightEdges(root);  
    int expected = (int)pow(2, h + 1) - 1;  
    return total == expected;  
}
```

Meaning:

1. Count the real nodes.
2. Find the height.
3. Calculate how many nodes a full tree of that height should have.
4. Compare actual nodes with expected nodes.

Example:

```
Height = 2
Expected full nodes =  $2^{(2 + 1)} - 1 = 7$ 
```

If actual nodes = 7, the tree is full.

If actual nodes is not 7, the tree is not full.

39. Strict Tree Checker

A strict tree allows only:

- 0 children
- 2 children

Code:

```
bool isStrict(Node* root) {
    if (root == nullptr) return true;

    if (root->left == nullptr && root->right == nullptr) {
        return true;
    }

    if (root->left != nullptr && root->right != nullptr) {
        return isStrict(root->left) && isStrict(root->right);
    }

    return false;
}
```

Explanation:

Case	Meaning	Return
<code>root == nullptr</code>	empty position	true
no left and no right	leaf node	true
left and right both exist	valid so far; check subtrees	recursive result
only one child exists	breaks strict rule	false

Memory trick:

```
Empty -> okay
Leaf -> okay
Two children -> check both sides
One child -> not okay
```

40. Complete Tree Checker Using Indices

A complete tree has no array gaps.

Instead of actually creating an array, we assign array-style indices during recursion.

Rules:

```
root index = 1
left child index = 2 * i
right child index = 2 * i + 1
```

Code:

```
bool isComplete(Node* root, int index, int totalNodes) {
    if (root == nullptr) return true;

    if (index > totalNodes) return false;

    return isComplete(root->left, 2 * index, totalNodes) &&
           isComplete(root->right, 2 * index + 1, totalNodes);
}
```

Meaning:

- If the current position is empty, it does not break the rule by itself.

- If a real node gets an index greater than the total number of nodes, then a gap must exist.
- So the tree is not complete.

Why does `index > totalNodes` mean not complete?

Because in a complete tree with `N` nodes, valid filled positions should be:

1, 2, 3, ..., N

If a node is placed at position 7 while the tree has only 5 nodes, then positions before 7 must be missing.

That means there is a gap.

Part J: Complete Beginner C++ Program

41. Full Program

```
#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

struct Node {
    char data;
    Node* left;
    Node* right;

    Node(char val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

int countNodes(Node* root) {
    if (root == nullptr) {
        return 0;
    }

    return 1 + countNodes(root->left) + countNodes(root->right);
}
```

```

int heightEdges(Node* root) {
    if (root == nullptr) {
        return -1;
    }

    return 1 + max(heightEdges(root->left), heightEdges(root->right));
}

bool isFullPacked(Node* root) {
    int total = countNodes(root);
    int h = heightEdges(root);
    int expected = (int)pow(2, h + 1) - 1;

    return total == expected;
}

bool isStrict(Node* root) {
    if (root == nullptr) {
        return true;
    }

    // Leaf node: 0 children
    if (root->left == nullptr && root->right == nullptr) {
        return true;
    }

    // Node with 2 children
    if (root->left != nullptr && root->right != nullptr) {
        return isStrict(root->left) && isStrict(root->right);
    }

    // Node with exactly 1 child
    return false;
}

bool isComplete(Node* root, int index, int totalNodes) {
    if (root == nullptr) {
        return true;
    }

    if (index > totalNodes) {
        return false;
    }

    return isComplete(root->left, 2 * index, totalNodes) &&
        isComplete(root->right, 2 * index + 1, totalNodes);
}

```

```

int main() {
    /*
        A
       /
      B  C
     /
    D

    This tree is:
    - complete
    - not strict
    - not full
    */

    Node* root = new Node('A');
    root->left = new Node('B');
    root->right = new Node('C');
    root->left->left = new Node('D');

    int total = countNodes(root);

    cout << "Total nodes: " << total << endl;
    cout << "Height: " << heightEdges(root) << endl;
    cout << "Full packed? " << (isFullPacked(root) ? "Yes" : "No") << endl;
    cout << "Strict? " << (isStrict(root) ? "Yes" : "No") << endl;
    cout << "Complete? " << (isComplete(root, 1, total) ? "Yes" : "No") << endl;

    return 0;
}

```

42. Expected Output

```

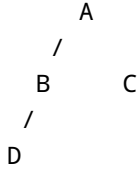
Total nodes: 4
Height: 2
Full packed? No
Strict? No
Complete? Yes

```

Part K: Dry Run of the Complete Program

43. Tree Created by the Program

The program creates this tree:



44. Counting Total Nodes

Nodes are:

A, B, C, D

So:

Total nodes = 4

45. Finding Height

Longest path:

A -> B -> D

This path has 2 edges.

So:

Height = 2

46. Checking Full Packed

Formula:

$$N = 2^{(H + 1)} - 1$$

Substitute height 2:

$$\text{Expected nodes} = 2^{(2 + 1)} - 1$$

$$\text{Expected nodes} = 2^3 - 1$$

$$\text{Expected nodes} = 8 - 1$$

$$\text{Expected nodes} = 7$$

Actual nodes:

4

Since:

$$4 \neq 7$$

The tree is not full.

So:

Full packed? No

47. Checking Strict

Check each node:

Node	Children	Strict?
A	B and C	valid
B	D only	not valid
C	none	valid
D	none	valid

Node B has exactly one child.

That breaks strictness.

So:

Strict? No

48. Checking Complete

Assign array-style indices:

```
A = 1  
B = 2  
C = 3  
D = 4
```

Total nodes = 4.

No node has index greater than 4.

Positions filled are:

1, 2, 3, 4

No gaps exist.

So:

Complete? Yes

Part L: Complexity Analysis

49. Time Complexity

countNodes

Visits every node once.

Time = $O(N)$

heightEdges

Visits every node once.

Time = $O(N)$

isFullPacked

Calls `countNodes` and `heightEdges`.

Each is $O(N)$.

So:

$O(N) + O(N) = O(N)$

isStrict

May visit every node once.

Time = $O(N)$

isComplete

May visit every node once.

Time = $O(N)$

50. Space Complexity

These functions are recursive, so they use the system stack.

The maximum stack depth depends on the height of the tree.

Extra space = $O(H)$

Where:

- H = height of the tree

For a balanced tree, H is smaller.

For a skewed tree, H can be close to N .

Worst case:

Extra space = $O(N)$

51. Complexity Summary Table

Function	Time complexity	Extra space
countNodes	$O(N)$	$O(H)$
heightEdges	$O(N)$	$O(H)$
isFullPacked	$O(N)$	$O(H)$
isStrict	$O(N)$	$O(H)$
isComplete	$O(N)$	$O(H)$

Part M: Common Beginner Mistakes

52. Mistake 1: Using Strict and Complete as the Same Word

Wrong idea:

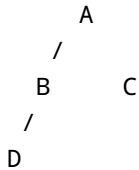
Strict and complete mean the same thing.

Correct idea:

They are different rules.

Strict = every node has 0 or 2 children.
Complete = no array gaps.

Example:



This tree is complete but not strict.

53. Mistake 2: Thinking Complete Means Every Level Is Full

Wrong idea:

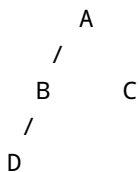
A complete tree must have every level fully filled.

Correct idea:

A complete tree can have a partially filled last level.

The last level must only be filled from left to right.

Example:



This is complete even though the last level is not full.

54. Mistake 3: Checking Completeness by Node Count Only

Wrong idea:

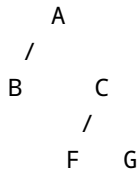
If the tree has a certain number of nodes, it must be complete.

Correct idea:

Node count alone cannot reveal gaps.

You must check positions.

Example:



This tree has 5 nodes.

But indices are:

1, 2, 3, 6, 7

There are gaps at positions 4 and 5.

So it is not complete.

55. Mistake 4: Calling Every Strict Tree Complete

Wrong idea:

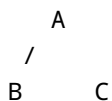
If a tree is strict, it must be complete.

Correct idea:

Strict only checks child count.

A strict tree can still have array gaps.

Example:





This tree is strict, but not complete.

56. Mistake 5: Confusing Full and Strict

Wrong idea:

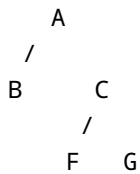
Full and strict mean the same thing.

Correct idea:

They are different.

Full = every level is completely packed.
 Strict = every node has 0 or 2 children.

Example of strict but not full:



It is strict because every node has 0 or 2 children.

It is not full because level 2 is not completely packed from left to right.

57. Mistake 6: Forgetting the Height Formula for Full Trees

Wrong formula:

$$N = 2^H - 1$$

Correct formula:

$$N = 2^{(H + 1)} - 1$$

Why?

Because height starts at 0.

If height is 2, there are 3 levels:

0, 1, 2

So we use $H + 1$.

Part N: Final Summary

58. Main Difference Table

Type	Simple rule	What it checks
Strict	Every node has 0 or 2 children	child count
Full	Every level is completely packed	maximum nodes for height
Complete	No gaps in level-order array layout	array positions

59. Relationship Table

Statement	True or false?	Reason
Every full tree is complete	True	A fully packed tree has no gaps.
Every complete tree is full	False	Last level may be partially filled.
Every strict tree is complete	False	Strict trees can have array gaps.
Every complete tree is strict	False	A complete tree can have a one-child node.

Statement	True or false?	Reason
A full packed tree is strict	True for non-empty full packed binary trees	Internal nodes have 2 children, leaves have 0.

60. Final Memory Tricks

Strict = 0 or 2 children.
 Full = every level packed.
 Complete = no gaps.

Another memory trick:

Strict asks: How many children does each node have?
 Full asks: Is the tree totally filled for its height?
 Complete asks: Are the array positions filled without gaps?

Shortest summary:

Strict = child rule.
 Full = packing rule.
 Complete = gap rule.

Part O: Practice Questions

61. Practice Question 1

Is this tree strict?

```

  A
 /
B  C

```

Answer:

Yes.

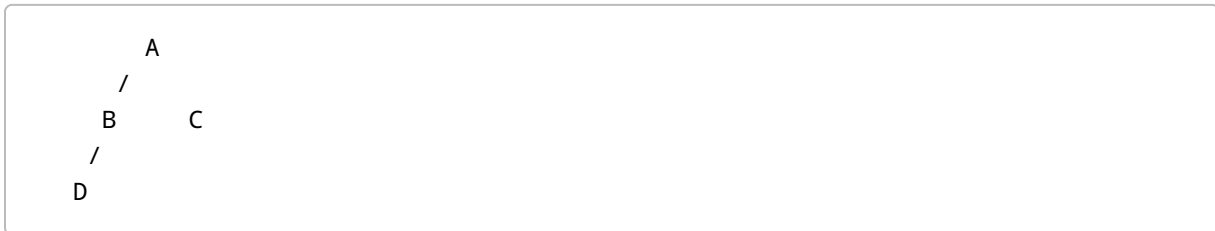
Why?

A has 2 children. B and C have 0 children.

All nodes have 0 or 2 children.

62. Practice Question 2

Is this tree complete?



Answer:

Yes.

Why?

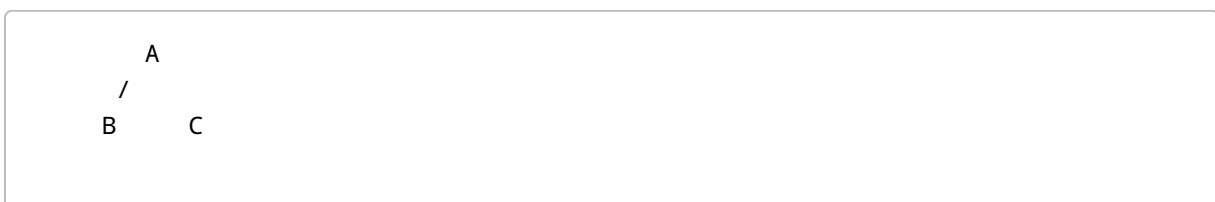
Array positions are:

```
A = 1
B = 2
C = 3
D = 4
```

No gaps exist.

63. Practice Question 3

Is this tree strict?



/
D

Answer:

No.

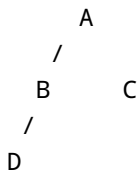
Why?

B has exactly one child.

A node with one child breaks strictness.

64. Practice Question 4

Is this tree full?



Answer:

No.

Why?

Height = 2.

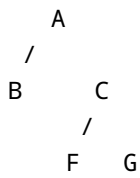
A full tree of height 2 must have:

$$2^{(2 + 1)} - 1 = 7 \text{ nodes}$$

This tree has only 4 nodes.

65. Practice Question 5

Is this tree strict and complete?



Answer:

Strict: Yes.

Complete: No.

Why?

Strict:

- A has 2 children.
- B has 0 children.
- C has 2 children.
- F and G have 0 children.

Complete:

A = 1
 B = 2
 C = 3
 F = 6
 G = 7

Positions 4 and 5 are missing.

So it is not complete.

66. Practice Question 6

A full binary tree has height 3. How many nodes does it have?

Formula:

$$N = 2^{(H + 1)} - 1$$

Substitute:

$N = 2^{(3 + 1)} - 1$
 $N = 2^4 - 1$
 $N = 16 - 1$
 $N = 15$

Answer:

15 nodes

67. Practice Question 7

A tree has 5 nodes, and one node has index 7 using array-style indexing. Can the tree be complete?

Answer:

No.

Why?

For a complete tree with 5 nodes, valid positions should be:

1, 2, 3, 4, 5

If a real node has index 7, then there must be missing positions before it.

So the tree has gaps.

68. Phase 8 Final Checklist

Before moving to Phase 9, make sure you can answer these:

1. What is a full binary tree in this curriculum?
2. What formula gives the number of nodes in a full binary tree of height H ?
3. Why does the formula use $H + 1$?
4. What is a complete binary tree?
5. What does "last level filled left to right" mean?
6. Why is every full tree complete?
7. Why is every complete tree not necessarily full?
8. What is a strict binary tree?
9. Why can a strict tree be incomplete?

10. Why can a complete tree be non-strict?
11. How does the index method check completeness?
12. What does `index > totalNodes` mean in the complete checker?
13. How does `isStrict` detect a one-child node?
14. How does `isFullPacked` use height and node count?
15. What are the four strict/complete cases?

If you can answer these, you understand Phase 8 well.

69. One-Sentence Phase 8 Summary

Phase 8 teaches that **strict**, **full**, and **complete** binary trees are different ideas: strict means every node has 0 or 2 children, full means every level is completely packed, and complete means there are no gaps when the tree is stored level by level.